

# Scalable Parallel-in-Time Integration

## Acoustics in Expanding Volumes

Nathan Chapman<sup>1</sup>, and Andy Piacsek<sup>2</sup>

<sup>1</sup>Department of Computer Science, Central Washington University

<sup>2</sup>Department of Physics, Central Washington University

Innovations in Computational Acoustics - ASA 2024

# Work in Progress

- Work in progress
- Should be finished and available in the Spring!

# The Equation of Motion and Causality

- The equations of motion (EoM) are at the heart of physics

## The Equations of Motion

$$\sum_i \vec{F}_i = m\vec{a}$$



# The Equation of Motion and Causality

- The equations of motion (EoM) are at the heart of physics

## The Equations of Motion

$$\sum_i \vec{F}_i = m\vec{a}$$

- Numerical methods have been sequential to follow causality
- 
-

# The Equation of Motion and Causality

- The equations of motion (EoM) are at the heart of physics

## The Equations of Motion

$$\sum_i \vec{F}_i = m\vec{a}$$

- Numerical methods have been sequential to follow causality
- These methods haven't been able to benefit from parallelism
-

# The Equation of Motion and Causality

- The equations of motion (EoM) are at the heart of physics

## The Equations of Motion

$$\sum_i \vec{F}_i = m\vec{a}$$

- Numerical methods have been sequential to follow causality
- These methods haven't been able to benefit from parallelism
- But what if they could?

# Acoustics in Expanding Volumes

Wave  $\psi(t, \vec{x})$  defined by its density  $n$  and phase  $\theta$  as

$$\psi(t, \vec{x}) = \sqrt{n_0 + \Delta n(t, \vec{x})} e^{i(\theta_0 + \Delta \theta(t, \vec{x}))}$$

Wave Equation in Expanding Volume

$$\partial_t^2 \theta - \frac{\dot{a}}{a} \partial_t \theta - ac_0^2 \nabla^2 \theta = 0$$

# Acoustics in Expanding Volumes

Wave  $\psi(t, \vec{x})$  defined by its density  $n$  and phase  $\theta$  as

$$\psi(t, \vec{x}) = \sqrt{n_0 + \Delta n(t, \vec{x})} e^{i(\theta_0 + \Delta\theta(t, \vec{x}))}$$

Wave Equation in Expanding Volume

$$\overbrace{\partial_t^2 \theta - \frac{\dot{a}}{a} \partial_t \theta - ac_0^2 \nabla^2 \theta = 0} \quad \underbrace{\tilde{\theta}_{\vec{k}} = \mathcal{F}(\Delta\theta)(t, \vec{k}) \Rightarrow}_{\text{Spectral decomposition}}$$



# Acoustics in Expanding Volumes

Wave  $\psi(t, \vec{x})$  defined by its density  $n$  and phase  $\theta$  as

$$\psi(t, \vec{x}) = \sqrt{n_0 + \Delta n(t, \vec{x})} e^{i(\theta_0 + \Delta\theta(t, \vec{x}))}$$

Wave Equation in Expanding Volume

$$\partial_t^2 \theta - \frac{\dot{a}}{a} \partial_t \theta - ac_0^2 \nabla^2 \theta = 0$$

$$\begin{array}{c} \tilde{\theta}_{\vec{k}} = \mathcal{F}(\Delta\theta)(t, \vec{k}) \\ \Rightarrow \\ \text{Spectral decomposition} \end{array}$$

ODE for each wave number

$$\partial_t^2 \tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a} \partial_t \tilde{\theta}_{\vec{k}} - ac_0^2 (0)^2 \tilde{\theta}_{\vec{k}} = 0$$

$$\partial_t^2 \tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a} \partial_t \tilde{\theta}_{\vec{k}} - ac_0^2 (1)^2 \tilde{\theta}_{\vec{k}} = 0$$

$$\vdots$$

$$\partial_t^2 \tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a} \partial_t \tilde{\theta}_{\vec{k}} - ac_0^2 k^2 \tilde{\theta}_{\vec{k}} = 0$$

$$\vdots$$

$$\partial_t^2 \tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a} \partial_t \tilde{\theta}_{\vec{k}} - ac_0^2 k_c^2 \tilde{\theta}_{\vec{k}} = 0$$

Similar to the method of lines

# Parallel-in-time Integration and the Parareal Algorithm

- “Parallel-in-time integration”  $\approx$  “Solving IVP in parallel”



# Parallel-in-time Integration and the Parareal Algorithm

- “Parallel-in-time integration”  $\approx$  “Solving IVP in parallel”
- Several algorithms
  - Parareal
  - Multigrid Reduction in Time (MGRIT)
  - Parallel Full Approximation Scheme in Space and Time (PFASST)
  - and more
-

# Parallel-in-time Integration and the Parareal Algorithm

- “Parallel-in-time integration”  $\approx$  “Solving IVP in parallel”
- Several algorithms
  - Parareal
  - Multigrid Reduction in Time (MGRIT)
  - Parallel Full Approximation Scheme in Space and Time (PFASST)
  - and more
- The 4 cores steps of the Parareal Algorithm
  1. Prepare the subproblems
  2. Solve each subproblem in parallel
  3. Correct
  4. Iterate

# The Parareal Algorithm - Subproblem Preperation

1. Given an initial value problem  $P$

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t) \quad u(0) = u_0, \partial_t u(0) = v_0 \quad D = [T, T + \Delta T]$$

- 2.

- 3.

- 4.

- 5.

# The Parareal Algorithm - Subproblem Preperation

1. Given an initial value problem  $P$

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t) \quad u(0) = u_0, \partial_t u(0) = v_0 \quad D = [T, T + \Delta T]$$

2. Choose number of subproblems  $N$  (suggest number of compute cores)

- 3.

- 4.

- 5.

# The Parareal Algorithm - Subproblem Preperation

1. Given an initial value problem  $P$

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t) \quad u(0) = u_0, \partial_t u(0) = v_0 \quad D = [T, T + \Delta T]$$

2. Choose number of subproblems  $N$  (suggest number of compute cores)
3. Partition time domain for  $p = 0, 1, \dots, N - 1$

$$D = [T, T + \Delta T] \rightarrow D_p = [T + \frac{p}{N}\Delta T, T + \frac{p+1}{N}\Delta T] = [T_p, T_{p+1}]$$

- 4.

- 5.

# The Parareal Algorithm - Subproblem Preperation

1. Given an initial value problem  $P$

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t) \quad u(0) = u_0, \partial_t u(0) = v_0 \quad D = [T, T + \Delta T]$$

2. Choose number of subproblems  $N$  (suggest number of compute cores)
3. Partition time domain for  $p = 0, 1, \dots, N - 1$

$$D = [T, T + \Delta T] \rightarrow D_p = [T + \frac{p}{N}\Delta T, T + \frac{p+1}{N}\Delta T] = [T_p, T_{p+1}]$$

4. Choose coarse propagator  $\mathcal{C}_0$  to get initial solution values  $\{u_p^0\}_p, \{v_p^0\}_p$
- 5.



# The Parareal Algorithm - Subproblem Preperation

1. Given an initial value problem  $P$

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t) \quad u(0) = u_0, \partial_t u(0) = v_0 \quad D = [T, T + \Delta T]$$

2. Choose number of subproblems  $N$  (suggest number of compute cores)

3. Partition time domain for  $p = 0, 1, \dots, N - 1$

$$D = [T, T + \Delta T] \rightarrow D_p = [T + \frac{p}{N}\Delta T, T + \frac{p+1}{N}\Delta T] = [T_p, T_{p+1}]$$

4. Choose coarse propagator  $\mathcal{C}_0$  to get initial solution values  $\{u_p^0\}_p, \{v_p^0\}_p$

5. Problem  $P^0 \equiv \mathcal{L}(\dots) = f(t) \quad \underbrace{u(T_p) = u_p^0, \partial_t u(T_i) = v_p^0}_{\text{Subproblem initial values}} \quad \underbrace{D_p = [T_p, T_{p+1}]}_{\text{Subproblem domain}}$

# The Parareal Algorithm - Parallel Propagation

1. Choose coarse propagator  $\mathcal{C}$ 
  - For example, RK2 with a large time-step
- 2.
- 3.
- 4.

# The Parareal Algorithm - Parallel Propagation

1. Choose coarse propagator  $\mathcal{C}$ 
  - For example, RK2 with a large time-step
2. Choose fine propagator  $\mathcal{F}$ 
  - For example, Velocity-Verlet with a small time-step
- 3.
- 4.

# The Parareal Algorithm - Parallel Propagation

1. Choose coarse propagator  $\mathcal{C}$ 
  - For example, RK2 with a large time-step
2. Choose fine propagator  $\mathcal{F}$ 
  - For example, Velocity-Verlet with a small time-step
3. On  $i$ -th iteration, use  $\mathcal{C}$  to solve each subproblem  $p$  in parallel
  - coarse solution is  $\mathcal{C}u_p^i$
- 4.

# The Parareal Algorithm - Parallel Propagation

1. Choose coarse propagator  $\mathcal{C}$ 
  - For example, RK2 with a large time-step
2. Choose fine propagator  $\mathcal{F}$ 
  - For example, Velocity-Verlet with a small time-step
3. On  $i$ -th iteration, use  $\mathcal{C}$  to solve each subproblem  $p$  in parallel
  - coarse solution is  $\mathcal{C}u_p^i$
4. Use  $\mathcal{F}$  to solve each subproblem  $p$  in parallel
  - fine solution is  $\mathcal{F}u_p^i$

# The Parareal Algorithm - Corrections

- Previous main coarse solution is “corrected”
- 
-

# The Parareal Algorithm - Corrections

- Previous main coarse solution is “corrected”
- Correctors:  $\eta_p^i = \mathcal{F}u_p^i(T_{p+1}) - \mathcal{C}u_p^i(T_{p+1})$ 
  - Correct based on last value in each subproblem
-

# The Parareal Algorithm - Corrections

- Previous main coarse solution is “corrected”
- Correctors:  $\eta_p^i = \mathcal{F}u_p^i(T_{p+1}) - \mathcal{C}u_p^i(T_{p+1})$ 
  - Correct based on last value in each subproblem
- Correction:  $u_{p+1}^i(T_{p+1}) = u_p^i(T_{p+1}) + \eta_p^i$ 
  - Must be done sequentially



# The Parareal Algorithm - Iterate

- Iteration creates new subproblems based on corrected initial values, and repeats

$$P^i \equiv \mathcal{L}(\dots) = f(t) \quad \underbrace{u(T_p) = u_p^i, \partial_t u(T_i) = v_p^i}_{\text{Subproblem initial values}} \quad \underbrace{D_p = [T_p, T_{p+1}]}_{\text{Subproblem domain}}$$



# The Parareal Algorithm - Iterate

- Iteration creates new subproblems based on corrected initial values, and repeats

$$P^i \equiv \mathcal{L}(\dots) = f(t) \quad \underbrace{u(T_p) = u_p^i, \partial_t u(T_i) = v_p^i}_{\text{Subproblem initial values}} \quad \underbrace{D_p = [T_p, T_{p+1}]}_{\text{Subproblem domain}}$$

- Loop until some stopping criteria is met
  - For example, iterations converge  $|u_p^i - u_p^{i-1}| < \varepsilon \quad \forall p \leq N$



# The Parareal Algorithm - Iterate

- Iteration creates new subproblems based on corrected initial values, and repeats

$$P^i \equiv \mathcal{L}(\dots) = f(t) \quad \underbrace{u(T_p) = u_p^i, \partial_t u(T_i) = v_p^i}_{\text{Subproblem initial values}} \quad \underbrace{D_p = [T_p, T_{p+1}]}_{\text{Subproblem domain}}$$

- Loop until some stopping criteria is met
  - For example, iterations converge  $|u_p^i - u_p^{i-1}| < \varepsilon \quad \forall p \leq N$
- Converges in **at most**  $N$  iterations
-

# The Parareal Algorithm - Iterate

- Iteration creates new subproblems based on corrected initial values, and repeats

$$P^i \equiv \mathcal{L}(\dots) = f(t) \quad \underbrace{u(T_p) = u_p^i, \partial_t u(T_i) = v_p^i}_{\text{Subproblem initial values}} \quad \underbrace{D_p = [T_p, T_{p+1}]}_{\text{Subproblem domain}}$$

- Loop until some stopping criteria is met
  - For example, iterations converge  $|u_p^i - u_p^{i-1}| < \varepsilon \quad \forall p \leq N$
- Converges in **at most**  $N$  iterations
- End result is same as applying the fine solver directly

# GPU Computing

- CPU parallelism
  - Well supported via OpenMP, MPI, etc.
  - ~10 cores  $\rightarrow$  ~10 subproblems
  - Not very accurate
- 
-

# GPU Computing

- CPU parallelism
  - Well supported via OpenMP, MPI, etc.
  - ~10 cores  $\rightarrow$  ~10 subproblems
  - Not very accurate
- These calculations only involve arithmetic!
-

# GPU Computing

- CPU parallelism
  - Well supported via OpenMP, MPI, etc.
  - $\sim 10$  cores  $\rightarrow \sim 10$  subproblems
  - Not very accurate
- These calculations only involve arithmetic!
- GPU parallelism
  - Well supported by CUDA, OpenACC, etc.
  - 15,000+ cores  $\rightarrow$  15,000+ subproblems
  - **Very accurate**

# Distributed Computing

- 1 machine  $\Rightarrow$  1 wave number at a time
- 
- 
-



# Distributed Computing

- 1 machine  $\Rightarrow$  1 wave number at a time
- What if we had multiple machines?
- 
-

# Distributed Computing

- 1 machine  $\Rightarrow$  1 wave number at a time
- What if we had multiple machines?
- Distributed computing e.g. MPI  $\Rightarrow$  **all** wave numbers at the *same time*
-

# Distributed Computing

- 1 machine  $\Rightarrow$  1 wave number at a time
- What if we had multiple machines?
- Distributed computing e.g. MPI  $\Rightarrow$  **all** wave numbers at the *same time*
- Can scale to clusters and supercomputers!

# Some Basic Results

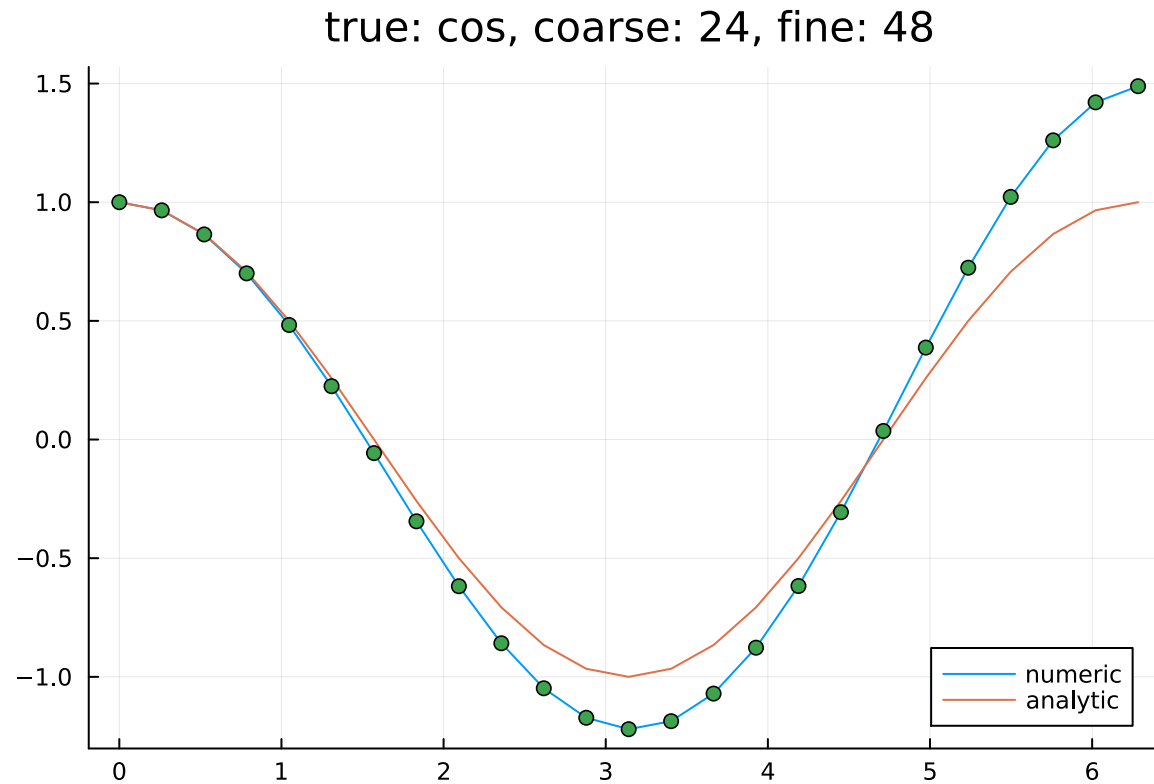


Figure 1: Local CPU-based solution to  $u''(t) = -u(t)$ ,  $u(0) = 1$ ,  $u'(0) = 0$  evaluated on 24 threads

# Conclusion

- Equations of motion are no longer bound to sequential solvers
- Certain problems can be divided and conquered
- “Doubly parallel” problems can benefit from
  - local parallelism
  - distributed parallelism
  - High-performance systems
- One day soon, used to model acoustics in expanding volumes

## Thank you for your time

- Julia code available on GitHub →
- Let's collaborate!
- [nathaniel.chapman@cwu.edu](mailto:nathaniel.chapman@cwu.edu)
- [nathanwchapman.com](http://nathanwchapman.com)

